



Sualeh Abbas 10303

Ayesha Shahid 10278

Lab 2: Implementing Decision Instructions in RISC V Assembly Language

Task 1:

Type the codes in Listings 3 and 4 in the assembler editor and then in the simulator look at their binary:

- The beq and bne instructions are stored in the SB format with their 32-bit binary. Look it up and try to find out what values their different fields contain.
- Looking at the binary formats, find out how do the labels are stored in the 32-bit instruction. Compare the label values stored inside the instruction to the code and try to guess how the assembler stores the jump addresses inside the instruction. (Hint: it uses a PC-relative scheme) Refer to following figures (from book page 119) to see encoding of instructions.

Note: To test listing 4, ld will be replaced by lw and for computation of address, offset should be obtained by $i*4$ i.e., shift left by 2 instead of

For Listing 3

```
1  if ( i == j )
2      f = g + h;
3  else
4      f = g - h;
5  // code after if/else goes here
6
7
8
9  //assuming that variables f to j are in registers x19-x23
10 bne x22, x23, Else
11 add x19, x20, x21
12 beq x0, x0, Exit //un conditional jump
13 Else: sub x19, x20, x21
14
15 Exit: # the code after if/else goes here
```

Listing 3: If statement

BEQ

01000	00000	00000	000	01000	1100011
-------	-------	-------	-----	-------	---------

BNE

01100	10110	1011	001	00000	1100011
-------	-------	------	-----	-------	---------



For listing 4

```
1  while (save[i] == k)
2      i += 1;
3
4
5  // assuming i and k in x22 and x24, and the base address of Save in x25
6  Loop: slli x10, x22, 3 // Temp reg x10 = i * 8
7        add x10, x10, x25 // x10 = address of save[i]
8        ld x9, 0(x10) // Temp reg x9 = save[i]
9        bne x9, x24, Exit // go to Exit if save[i] != k
10       addi x22, x22, 1 // i = i + 1
11       beq x0, x0, Loop // go to Loop
12  Exit:
13
```

Listing 4: while loop

BEQ

10100	00000	00000	000	10100	1100011
-------	-------	-------	-----	-------	---------

BNE

01100	01001	11000	001	01100	1100011
-------	-------	-------	-----	-------	---------

Task 2:

The switch statement is similar to if/else statements. Write the equivalent RISC-V assembly code for Listing 5.

Assume that the variables x, a, b & c are signed integers and stored in x20, x21, x22, x23 respectively. Make sure to first assign suitable values for b and c first.

```
1 switch (x) {
2     case 1:
3         a = b+c;
4         break;
5     case 2:
6         a = b-c;
7         break;
8     case 3:
9         a = b * 2;
10        break;
11    case 4:
12        a = b / 2;
13        break;
14    default:
15        a = 0;
16 }
```

Listing 5: task 1b.c

Code:

```
.text
.globl
main:
    li x1, 1 #case1
    li x2, 2 #case2
    li x3, 3 #case3
    li x4, 4 #case4

    li x22, 20 # b
    li x23, 21 # c
    li x20, 1 #x
    li x24, 2 #integer value
    beq x20, x1, case1
    beq x20, x2, case2
    beq x20, x3, case3
    beq x20, x4, case4

    li x21, 0

case1:
    add x21, x22, x23
    beq x0, x0, exit
case2:
```



```
sub x21, x22, x23
beq x0, x0, exit
case3:
mul x21, x22, x24
beq x0, x0, exit
case4:
div x21, x22, x24
beq x0, x0, exit
exit:
```

Output:

Case 1:

x15 (a5) = 0x00000000	13	beq x20, x1, case1
x16 (a6) = 0x00000000	14	beq x20, x2, case2
x17 (a7) = 0x00000000	15	beq x20, x3, case3
x18 (s2) = 0x00000000	16	beq x20, x4, case4
x19 (s3) = 0x00000000	17	
x20 (s4) = 0x00000001	18	li x21, 0
x21 (s5) = 0x00000029	19	
x22 (s6) = 0x00000014	20	case1:
x23 (s7) = 0x00000015	21	add x21, x22, x23
x24 (s8) = 0x00000002	22	beq x0, x0, exit
x25 (s9) = 0x00000000	23	case2:
x26 (s10) = 0x00000000	24	sub x21, x22, x23
x27 (s11) = 0x00000000	25	beq x0, x0, exit
x28 (t3) = 0x00000000	26	case3:
	27	mul x21, x22, x24
	28	beq x0, x0, exit
	29	case4:

Case 2:

x16 (a6) = 0x00000000	10	li x20, 2 #x
x17 (a7) = 0x00000000	11	li x24, 2 #integer value
x18 (s2) = 0x00000000	12	
x19 (s3) = 0x00000000	13	beq x20, x1, case1
x20 (s4) = 0x00000002	14	beq x20, x2, case2
x21 (s5) = 0xFFFFFFFF	15	beq x20, x3, case3
x22 (s6) = 0x00000014	16	beq x20, x4, case4
x23 (s7) = 0x00000015	17	
x24 (s8) = 0x00000002	18	li x21, 0
x25 (s9) = 0x00000000	19	
x26 (s10) = 0x00000000	20	case1:
x27 (s11) = 0x00000000	21	add x21, x22, x23
x28 (t3) = 0x00000000	22	beq x0, x0, exit
	23	case2:
	24	sub x21, x22, x23
	25	beq x0, x0, exit



Case 3:

x16 (a6)	=	0x00000000	10	li x23, 21 # C
x17 (a7)	=	0x00000000	11	li x20, 3 #x
x18 (s2)	=	0x00000000	12	li x24, 2 #integer value
x19 (s3)	=	0x00000000	13	beq x20, x1, case1
x20 (s4)	=	0x00000003	14	beq x20, x2, case2
x21 (s5)	=	0x00000028	15	beq x20, x3, case3
x22 (s6)	=	0x00000014	16	beq x20, x4, case4
x23 (s7)	=	0x00000015	17	li x21, 0 #default case
x24 (s8)	=	0x00000002	18	case1:
x25 (s9)	=	0x00000000	19	add x21, x22, x23
x26 (s10)	=	0x00000000	20	beq x0, x0, exit
x27 (s11)	=	0x00000000	21	case2:
x28 (t3)	=	0x00000000	22	sub x21, x22, x23
x29 (t4)	=	0x00000000	23	beq x0, x0, exit
			24	case3:
			25	mul x21, x22, x24
			26	beq x0, x0, exit

Case 4:

x18 (s2)	=	0x00000000	19	add x21, x22, x23
x19 (s3)	=	0x00000000	20	beq x0, x0, exit
x20 (s4)	=	0x00000004	21	case2:
x21 (s5)	=	0x0000000A	22	sub x21, x22, x23
x22 (s6)	=	0x00000014	23	beq x0, x0, exit
x23 (s7)	=	0x00000015	24	case3:
x24 (s8)	=	0x00000002	25	mul x21, x22, x24
x25 (s9)	=	0x00000000	26	beq x0, x0, exit
x26 (s10)	=	0x00000000	27	case4:
x27 (s11)	=	0x00000000	28	div x21, x22, x24
x28 (t3)	=	0x00000000	29	beq x0, x0, exit
			30	exit:

Task 3:

Write the equivalent RISC-V assembly code for Listing 6. Assume that the variables `i` and `sum` are in `x22` and `x23`, while the array `a` located at address `0x200` is of 4-byte integers.

```
1 for (int i=0; i<10; i++)
2   a[i] = i;
3
4 for (int i=0; i<10; i++)
5   sum = sum+a[i];
```

Listing 6: Reduction Sum

Code:

```
.text
.globl
main:
    li t0, 0x200
    li x25, 10
    li x22, 0 #i=0
    li x26, 0
    li x23, 0 #sum=0
loop1:
    bge x22, x25, loop2
    slli x10, x22, 2
    add t1, t0, x10
    sw x22, 0(t1)
    addi x22, x22, 1
    j loop1

loop2:
    bge x26, x25, end
    slli x10, x26, 2
    add t1, t0, x10
    lw x24, 0(t1) # x24 = a[i]
    add x23, x23, x24 #sum= sum + a[i]
    addi x26, x26, 1
    j loop2

end:
```

Output:

ASM task3.s	Address	+0	+1	+2	+3
1 .text	0x00000230	00	00	00	00
2 .globl	0x0000022C	00	00	00	00
3 main:	0x00000228	00	00	00	00
4 li t0, 0x200	0x00000224	09	00	00	00
5 li x25, 10	0x00000220	08	00	00	00
6 li x22, 0 #i=0	0x0000021C	07	00	00	00
7 loop1:	0x00000218	06	00	00	00
8 bge x22, x25, end	0x00000214	05	00	00	00
9	0x00000210	04	00	00	00
10 slli x10, x22, 2	0x0000020C	03	00	00	00
11 add t1, t0, x10	0x00000208	02	00	00	00
12 sw x22, 0(t1)	0x00000204	01	00	00	00
13	0x00000200	00	00	00	00
14 addi x22, x22, 1					
15 j loop1					
16 end:					
17					
18					

x16 (a6) = 0x00000000	12 add t1, t0, x10
x17 (a7) = 0x00000000	13 sw x22, 0(t1)
x18 (s2) = 0x00000000	14 addi x22, x22, 1
x19 (s3) = 0x00000000	15 j loop1
x20 (s4) = 0x00000000	16
x21 (s5) = 0x00000000	17 loop2:
x22 (s6) = 0x0000000A	18 bge x26, x25, end
x23 (s7) = 0x0000002D	19 slli x10, x26, 2
x24 (s8) = 0x00000009	20 add t1, t0, x10
x25 (s9) = 0x0000000A	21 lw x24, 0(t1) # x24 = a[i]
x26 (s10) = 0x0000000A	22 add x23, x23, x24 #sum= sum + a[i]
x27 (s11) = 0x00000000	23 addi x26, x26, 1
x28 (t3) = 0x00000000	24 j loop2
x29 (t4) = 0x00000000	25
	26 end:
	27 j end

Task 4 (Challenge):

Translate the following C code to RISC-V assembly code. Use a minimum number of instructions. Assume that the values of a, b, i, and j are in registers x5, x6, x7, and x29, respectively. Also, assume that register x10 holds the base address of the array D.

```

1
2 for(i=0; i<a; i++)
3
4     1
5     for(i=0; i<a; i++)
6     for(j=0; j<b; j++)
7         D[4*j] = i + j;

```

Listing 7: Nested Loops



Code:

```
.text
.globl
main:
    li x5, 2    # assuming a = 2
    li x6, 3    # assuming b = 3
    li x10, 0x200 # base address of D in memory

    addi x7, x0, 0 # i = 0
i_loop:
    bge x7, x5, end # i >= a

    addi x29, x0, 0 # j = 0
j_loop:
    bge x29, x6, next_i # j >= b

    add x8, x7, x29    # x8 = i + j
    slli x11, x29, 4    # x11 = j * 16 (4*j * 4 bytes)
    add x11, x10, x11    # new calculated address (D[4*j])
    sw x8, 0(x11)        # D[4*j] = i + j
    addi x29, x29, 1    # j++
    j j_loop # repeat inner loop

next_i:
    addi x7, x7, 1    # i++
    j i_loop # repeat outer loop

end:
    j end
```

Output

7			0x00000220	00	00	00	00
8	<code>addi x7, x0, 0 # i = 0</code>		0x00000224	00	00	00	00
9	<code>i_loop:</code>		0x00000220	03	00	00	00
10	<code>bge x7,x5, end #i>=a</code>		0x0000021C	00	00	00	00
11			0x00000218	00	00	00	00
12	<code>addi x29, x0, 0 # j = 0</code>		0x00000214	00	00	00	00
13	<code>j_loop:</code>		0x00000210	02	00	00	00
14	<code>bge x29, x6, next_i #j>=b</code>		0x0000020C	00	00	00	00
15			0x00000208	00	00	00	00
16	<code>add x8, x7, x29 # x8 = i + j</code>		0x00000204	00	00	00	00
17	<code>slli x11, x29, 4 # x11 = j * 16 (4*j)</code>		0x00000200	01	00	00	00
18	<code>add x11, x11, x11 # new calculated add</code>						
x05 (t0)	= 0x00000002	5	<code>li x6, 3 #assuming b = 4</code>				
x06 (t1)	= 0x00000003	6	<code>li x10, 0x200 # base address of D in</code>				
x07 (t2)	= 0x00000002	7					
x08 (s0)	= 0x00000003	8	<code>addi x7, x0, 0 # i = 0</code>				
x09 (s1)	= 0x00000000	9	<code>i_loop:</code>				
x10 (a0)	= 0x00000200	10	<code>bge x7,x5, end #i>=a</code>				
x11 (a1)	= 0x00000220	11					
		12	<code>addi x29, x0, 0 # j = 0</code>				



Assessment Rubric

Lab 2: Implementing Decision Instructions in RISC V Assembly Language

Name: Ayesha Mohammad Shahid Syed Sualeh Abbas	Student ID: 10278 10303	Section: T4
--	-----------------------------------	--------------------

Points Distribution

	Task No.	LR 2 Code	LR 5 Results
In - Lab	Task 1	/0	/5
	Task 2	/10	/10
	Task 3	/15	/10
	Task 4	/20	/10
Total Points: 100		/45	/35
CLO Mapped		CLO2	

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission	/20	CLO 2

CLO	Total Points	Points Obtained
2	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab



Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.